

# We told the AI to think harder.

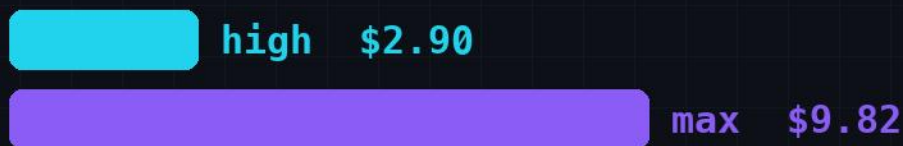
## Mostly, it just wrote more.

53 test runs of ArcKit on Claude Opus 5.5 and Sonnet 5.5, \$137 of recorded model time. Here's what the highest effort setting actually bought.

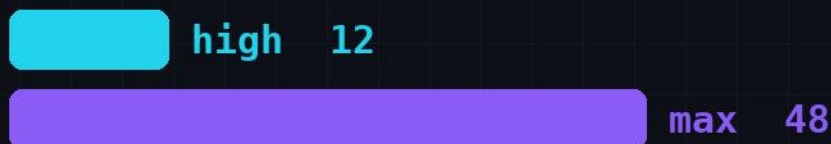
# Same requirements. Four times the wait.

One requirements document, Claude Opus 5.5, normal effort against max.

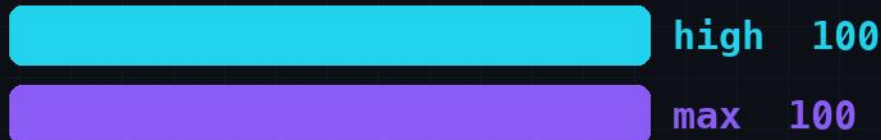
## Cost



## Minutes



## Requirements written (average)



# The real problem was the examples.

13 of the 55 ArcKit command templates we audited showed examples that contradicted their own instructions. At normal effort, the AI copies the example.

## Requirements template

The instruction: every requirement needs acceptance criteria.

The examples: 1 in 30 had them.

The result: every run left them off 36 to 61 requirements, even at max.

# Fix the example, not the effort.

With the templates fixed, at normal effort:

## Requirements

Every requirement complete in 4 of 4 runs, on both models.

Sonnet 5.5: mentions of acceptance criteria, 52 → 132.

## 12 other templates

Security actions now have owners. Runbooks have rollback steps.

The 'do nothing' option now has a cost.

# What we'd take from it

- 1 Check the template before raising the effort.**  
Including your own customised copies.
- 2 Choose the model, not the effort level.**  
In these tests it mattered more.
- 3 Measure cost and time, not just pass or fail.**  
A 48-minute wait is a cost too.

Full write-up, method and every cost: [arckit.org/articles](https://arckit.org/articles)